

CSCE 670 INFORMATION RETRIEVAL · MODULE 02 · LECTURE NOTES

The Index

The data structure that makes search possible.

MODULE OUTCOMES

- Explain the inverted index and what it stores.
- Explain the preprocessing decisions and their costs.
- Explain postings compression and why it is worth it.
- Explain index construction at scale.
- Build an index and measure it.

1 The structure

DICTIONARY	term -> (document frequency, pointer into the postings)
POSTINGS LIST	for each term, the documents containing it, in increasing document-id order: doc_id, term_frequency, [positions]
AND THE CHOICES	
positions	needed for phrase and proximity queries; they roughly triple the index size
frequencies	needed for any ranked retrieval at all (Module 03)
field info	which part of the document the term appeared in -- title, body, anchor text -- for Module 06's features
WHY SORTED BY DOCUMENT ID	
	intersecting two sorted lists is linear in their lengths, and compression works far better on an increasing sequence (section 3).

Sorted by document id is the decision everything else follows from — it makes intersection linear and it makes compression effective, at the same time and for the same reason. An index sorted by relevance or by frequency would support neither. **That structural insight is worth generalising:** choosing the order a structure is stored in frequently determines which operations are cheap, and it is a decision

made once.

THE INDEX TRADES SPACE AND BUILD TIME FOR QUERY TIME, WHICH IS THE RIGHT TRADE

Scanning every document for every query is $O(\text{collection})$ per query — which is unaffordable at any meaningful scale, since queries arrive constantly and the collection is large.

An inverted index makes a query cost proportional to the length of the matching postings lists — which for most terms is a tiny fraction of the collection, and which is the entire point of the structure.

And the cost is an index typically 10 to 40% of the collection's size (depending heavily on whether positions are stored), built once and then updated incrementally (§4).

Which is a good trade precisely because queries vastly outnumber indexing operations — and recognising which side of a read/write asymmetry you are on is the general lesson, applicable well beyond search: it is the same reasoning behind a database index, a materialised view, and a cache.

2 Preprocessing

- **Tokenisation.** What counts as a term — hyphenation, apostrophes, numbers, product identifiers, URLs, and non-space-delimited scripts (CSCE 638 Module 02's whole subject, and the same conclusion: it is a modelling decision rather than a detail).
- **Case folding.** Nearly always worth doing, and it loses 'US' versus 'us' and 'May' versus 'may' — which matters considerably for named entity queries and barely at all for topical ones, so the right answer depends on your query mix.
- **Stemming.** Improves recall and reduces precision — and aggressive stemmers conflate unrelated words ('organisation' and 'organ', 'university' and 'universe'), which is why lighter stemmers and plain lemmatisation are now generally preferred.
- **Stopword removal.** Once essential for space, and now mostly unnecessary since compression handles the common terms efficiently — and it breaks phrase queries like 'to be or not to be' entirely, which is a memorable and genuine failure.
- **And the index must apply the same transformations to queries as to documents — a mismatch here is a silent and complete failure for the affected terms, not a degradation. The query-document symmetry requirement is the one that causes real bugs:** a stemmer applied at index time and forgotten at query time makes an entire class of term unfindable, with no error anywhere.

3 Compression

GAP ENCODING

store differences rather than absolute ids:

17, 41, 42, 93 -> 17, 24, 1, 51

Frequent terms give small gaps, which is exactly where the space is being spent.

VARIABLE-BYTE

one byte per 7 bits of value, with a continuation flag in the top bit. Simple, fast to decode, and byte-aligned.

BIT-PACKED AND SIMD-FRIENDLY SCHEMES

pack a block of gaps at whatever width the largest one in the block needs. Much faster to decode than bit-by-bit codes, which is why they displaced Elias gamma and Golomb coding in practice despite compressing slightly worse.

AND THE REASON IT BUYS SPEED

the index is read from disk or from memory, and decoding is cheaper than the I/O it saves. A smaller index is a FASTER index, which is the counterintuitive part.

A smaller index is a faster index — because the bottleneck is data movement rather than decoding, which is CSCE 676 Module 01 §2's cost model arriving in this course. The speed argument is why **compression is not optional**: it is not a space-saving concession but a performance technique, and the schemes that won are the ones optimised for decode throughput rather than for compression ratio.

AND SKIP POINTERS MAKE INTERSECTION SUBLINEAR

Store occasional forward pointers into a postings list — so that **intersecting a short list against a long one can skip over the long list's irrelevant stretches** rather than walking them.

Which matters because query term frequencies are wildly uneven: intersecting a rare term's hundred postings against a common term's ten million is the normal case rather than the exception, and the naive merge walks all ten million.

And the right skip interval is approximately the square root of the list length, which balances the pointer storage overhead against the skipping gain — a classic square-root trade.

But skips interact badly with compression — **you cannot skip into the middle of a gap-encoded block without decoding it from the start**, since each gap depends on the previous value — **which is exactly why real formats store skips at block boundaries** and why the block size is a tuning parameter.

4 Building it

- **In memory**, for a collection that fits — a dictionary of lists, accumulated, then sorted and written out. Simple, and the right answer more often than people assume (CSCE 676 Module 10 §3).
- **Blocked sort-based indexing**, for a collection that does not fit: **index blocks that do fit, write each sorted block to disk, then merge the sorted runs — which is external merge sort**, and the merge is a single sequential pass.
- **Single-pass in-memory indexing**, which avoids needing a global term-to-id mapping by **writing a separate dictionary per block and merging the dictionaries too** — which matters when the vocabulary itself is too large to hold.
- **Distributed indexing**, which is a MapReduce partitioned by term (CSCE 676 Module 10) — **and the shuffle is the cost**, with the heavy-tailed term distribution producing exactly the skew that module warns about.
- **And incremental updates**, usually handled by **keeping a small in-memory index for recent changes and merging it into the main index periodically** — because **rebuilding a postings list is cheaper than updating a compressed one in place. The merge-rather-than-update conclusion recurs in storage engines generally**: a compressed sorted structure is cheap to read and expensive to modify, so you write a new one and merge.

AND WHAT TO MEASURE ABOUT AN INDEX

Size against the raw collection, with **the compression attributed per technique** rather than reported as a single total — which is **Project 1's requirement** and is what demonstrates that you know which technique bought what.

Build time, and whether it parallelises — since **a reindex is an operation you will perform repeatedly**, after every schema change and every preprocessing fix, and its cost determines how willing you are to experiment.

Query latency at the median and the 95th percentile, because **the tail is what users actually experience as 'the search is slow'** — and a good median with a bad tail is the common and misleading shape (Module 04 §4).

And the postings list length distribution, which is heavy-tailed (CSCE 676 Module 07 §1) and **determines which optimisations are worth implementing** — skips matter only because of that tail, and knowing its shape tells you how much.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Manning, Raghavan & Schütze, chapters 1 through 5 (free)	The whole module , with the construction algorithms and the compression codes derived.

Resource	What it covers for this module
Zobel & Moffat — Inverted files for text search engines (free)	§3 and §4 exhaustively — the reference, with the measurements behind every recommendation.
Lemire & Boytsov — Decoding billions of integers per second (free)	§3's modern schemes — why decode throughput beat compression ratio, with the numbers.
The Lucene index format documentation (free)	§1 through §4 as actually implemented — read the postings format description against this module.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Build an inverted index** with frequencies, on a collection of at least 100,000 documents.
- 2 **Add positions** and measure the size increase.
- 3 **Apply case folding, stemming, and stopwords removal** in turn, and measure the index size and the vocabulary after each.
- 4 **Deliberately apply a stemmer at index time only** and find the queries that now fail.
- 5 **Implement gap encoding** and measure the saving.
- 6 **Implement variable-byte** and measure both size and decode speed.
- 7 **Add skip pointers** at $\sqrt{n}$ intervals and measure intersection speed on a rare-plus-common term pair.
- 8 **Implement blocked sort-based indexing** on a collection larger than your memory.
- 9 **Attribute your total compression** to each technique separately.
- 10 **Report your query latency** at the median and the 95th percentile.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does an inverted index hold, and what are the storage choices?
- 2 Why sorted by document id?
- 3 What trade does the index make, and why is it the right one?
- 4 Name five preprocessing decisions and the cost of each.
- 5 Which preprocessing mistake is a silent failure?
- 6 Name three compression techniques and why gap encoding works.
- 7 Why does compression buy speed?

- 8 What do skip pointers achieve, and what is the right interval?
- 9 Why do skips conflict with compression?
- 10 Name four construction methods and the merge-not-update conclusion.